

**ΟΙΚΟΝΟΜΙΚΟ
ΠΑΝΕΠΙΣΤΗΜΙΟ
ΑΘΗΝΩΝ**



ATHENS UNIVERSITY
OF ECONOMICS
AND BUSINESS

Οικονομικό Πανεπιστήμιο Αθηνών, Τμήμα Πληροφορικής Παράλληλος Προγραμματισμός 2025-26

#2 Προγραμματιστική Εργασία (OpenMP)

Ανθίπη Φατσέα

AM: 3190209

E-mail: p3190209@aueb.gr

Περιγραφή Συστήματος

Όνομα υπολογιστή	anthippi
Επεξεργαστής	AMD Ryzen AI 7 350
Λειτουργικό Σύστημα	Windows 11 (MSYS2/UCRT64)
Πλήθος πυρήνων	8 Physical Cores / 16 Logical Processors
Μεταφραστής	gcc v15.2.0

Contents

Περιγραφή Συστήματος	1
Εισαγωγή.....	3
Σειριακή Εκτέλεση.....	3
Μέρος Α	4
Παραλληλοποίηση με χρήση Reduction	4
Τοπική Συσσώρευση και Χρήση Atomic No Reduction	4
Πειραματικά Αποτελέσματα και Σχολιασμός	5
Επίδραση του Μεγέθους Προβλήματος και Thread Overhead.....	6
Μέρος Β	8
Προσομοίωση Ακανόνιστου Φόρτου Εργασίας	8
Πειραματικά Αποτελέσματα	8
Μέρος Γ	11
Γραμμική Δημιουργία Tasks.....	11
Αναδρομική Διάσπαση	12
Πρόληψη Ανεξέλεγκτης Δημιουργίας Εργασιών	13
Πειραματικά Αποτελέσματα και Σύγκριση Μεθόδων	13
Παρατηρήσεις.....	14
Bonus.....	15
Ανάλυση και Τελικό Συμπέρασμα	15
Ποιοτική Υπεροχή	15
Τελικό Συμπέρασμα	16

Εισαγωγή

Η παρούσα εργασία πραγματεύεται την παραλληλοποίηση του υπολογισμού ενός ορισμένου ολοκληρώματος μέσω του προτύπου OpenMP, με στόχο τη βέλτιστη αξιοποίηση σύγχρονων πολυπύρηνων υπολογιστικών συστημάτων. Το μαθηματικό πρόβλημα αφορά την αριθμητική ολοκλήρωση μιας συνάρτησης $f(x)$ στο διάστημα $[a, b]$, προσεγγίζοντας το εμβαδόν μέσω του τραπεζοειδούς κανόνα για n υποδιαίρεσεις. Ο μαθηματικός τύπος που υπολογίζεται είναι ο εξής

$$\int_a^b f(x)dx \approx \frac{h}{2} \left[f(a) + f(b) + 2 \sum_{i=1}^{n-1} f(a + ih) \right], h = (b - a)/n$$

Σειριακή Εκτέλεση

Πριν από την εφαρμογή των παραλληλοποιημένων εκδόσεων με το OpenMP, πραγματοποιήθηκε η εκτέλεση του αρχικού σειριακού κώδικα integration.c.

Πίνακας: Αποτελέσματα Σειριακής Εκτέλεσης

Παράμετρος	Τιμή
Συνάρτηση	$f(x) = x^2$
Διάστημα ολοκλήρωσης	$[0, 10]$
Πλήθος υποδιαίρεσεων (N)	100.000.000
Αποτέλεσμα ολοκληρώματος	333.333333
Μέσος χρόνος (20 επαναλήψεις)	0,363480 δευτερόλεπτα

Παρατήρηση για το μέγεθος του προβλήματος

Εκτελέστηκε επίσης ο σειριακός κώδικας για πολύ μικρό αριθμό υποδιαίρεσεων ($N = 1.000$), σημειώνοντας μέσο χρόνο μόλις **0,000004 δευτερόλεπτα**. Όπως θα δειχθεί στα επόμενα κεφάλαια, ο χρόνος αυτός είναι ταχύτερος από την αντίστοιχη παράλληλη εκτέλεση του OpenMP ($\sim 0,000146s$), αποδεικνύοντας ότι για εξαιρετικά μικρά μεγέθη προβλήματος, το διαχειριστικό κόστος συντονισμού των νημάτων καθιστά τον παραλληλισμό ασύμφορο.

Μέρος Α

Σε αυτό το στάδιο, ο σειριακός κώδικας τροποποιήθηκε ώστε οι επαναλήψεις του βρόχου υπολογισμού να διαμοιραστούν στατικά στα διαθέσιμα νήματα. Η στρατηγική αυτή είναι ιδανική για τη συνάρτηση $f(x) = x^2$, καθώς ο υπολογιστικός φόρτος είναι ομοιόμορφα κατανεμημένος σε όλο το εύρος ορισμού. Για την αποφυγή συνθηκών συναγωνισμού (race conditions) κατά την ενημέρωση της κοινόχρηστης μεταβλητής του αθροίσματος, εφαρμόστηκαν δύο διαφορετικές προσεγγίσεις.

Παραλληλοποίηση με χρήση Reduction

Η πρώτη προσέγγιση χρησιμοποίησε τη ρήτρα `reduction(+:sum)` της οδηγίας `#pragma omp parallel for`. Με αυτή τη μέθοδο, το OpenMP αναλαμβάνει αυτόματα τη δημιουργία τοπικών αντιγράφων της μεταβλητής αθροίσματος για κάθε νήμα και τη συγχώνευσή τους στο τέλος της παράλληλης περιοχής.

Τοπική Συσσώρευση και Χρήση Atomic No Reduction

Στη δεύτερη προσέγγιση, ο συγχρονισμός υλοποιήθηκε χειροκίνητα. Κάθε νήμα χρησιμοποιεί μια τοπική μεταβλητή για να συσσωρεύσει το αποτέλεσμα του τμήματός του. Μόνο μετά την ολοκλήρωση όλων των επαναλήψεων που του αναλογούν, το νήμα προσθέτει το μερικό αποτέλεσμα στην καθολική μεταβλητή χρησιμοποιώντας την οδηγία `#pragma omp atomic`. Η μέθοδος αυτή ελαχιστοποιεί τη χρήση κλειδαριών, περιορίζοντας τον συγχρονισμό σε μία μόνο πράξη ανά νήμα. Ωστόσο, αν βάζαμε αυτή την κλειδαριά μέσα στον βρόχο, το πρόγραμμα θα καθυστερούσε υπερβολικά λόγω του συνεχούς κλειδώματος/ξεκλειδώματος.

```
#pragma omp parallel
{
    double local_sum = 0.0;

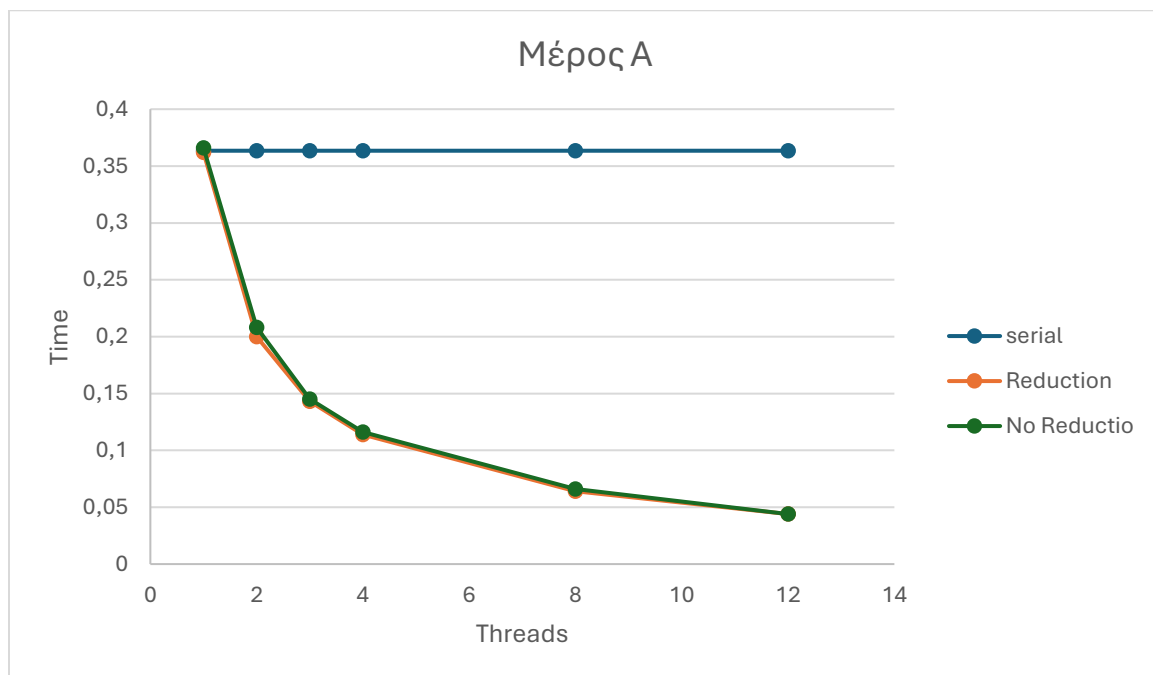
    #pragma omp for
    for (int i = 0; i < N; i++) {
        double x1 = a + i*h;
        double x2 = a + (i+1)*h;
        local_sum += (f(x1) + f(x2)) * h / 2.0;
    }

    #pragma omp atomic
    sum += local_sum;
}
```

Πειραματικά Αποτελέσματα και Σχολιασμός

Πραγματοποιήθηκαν μετρήσεις για 1, 2, 3, 4, 8 και 12 νήματα με σταθερό μέγεθος προβλήματος $N = 10^8$.

Threads	Reduction (sec)	No Reduction (sec)
1	0.362	0.366
2	0.200	0.208
3	0.143	0.145
4	0.114	0.116
8	0.064	0.066
12	0.044	0.044



Παρατηρήσεις

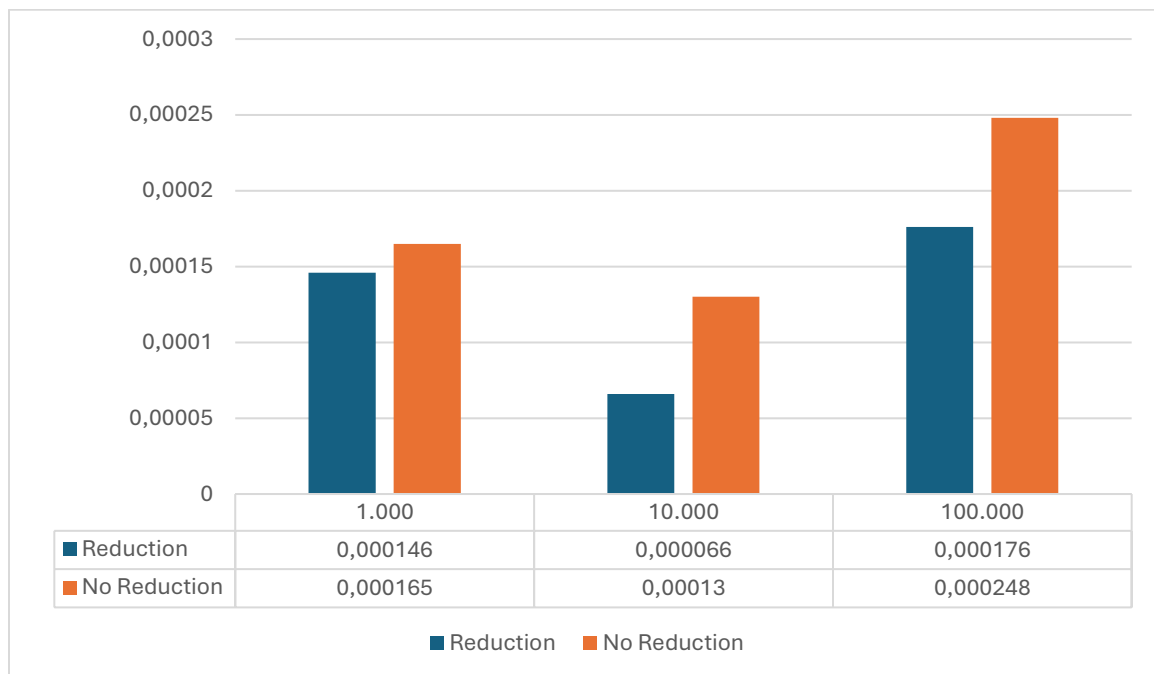
Οι δύο μέθοδοι εμφάνισαν σχεδόν πανομοιότυπους χρόνους. Αυτό οφείλεται στο γεγονός ότι στην έκδοση "No Reduction", η εντολή `atomic` τοποθετήθηκε εκτός του εσωτερικού βρόχου, εξαλείφοντας το `lock contention`, ακριβώς όπως κάνει εσωτερικά και το `reduction`. Επίσης παρατηρείται σταθερή μείωση του χρόνου εκτέλεσης καθώς αυξάνονται τα νήματα. Η βέλτιστη απόδοση σημειώνεται στα **8 νήματα**.

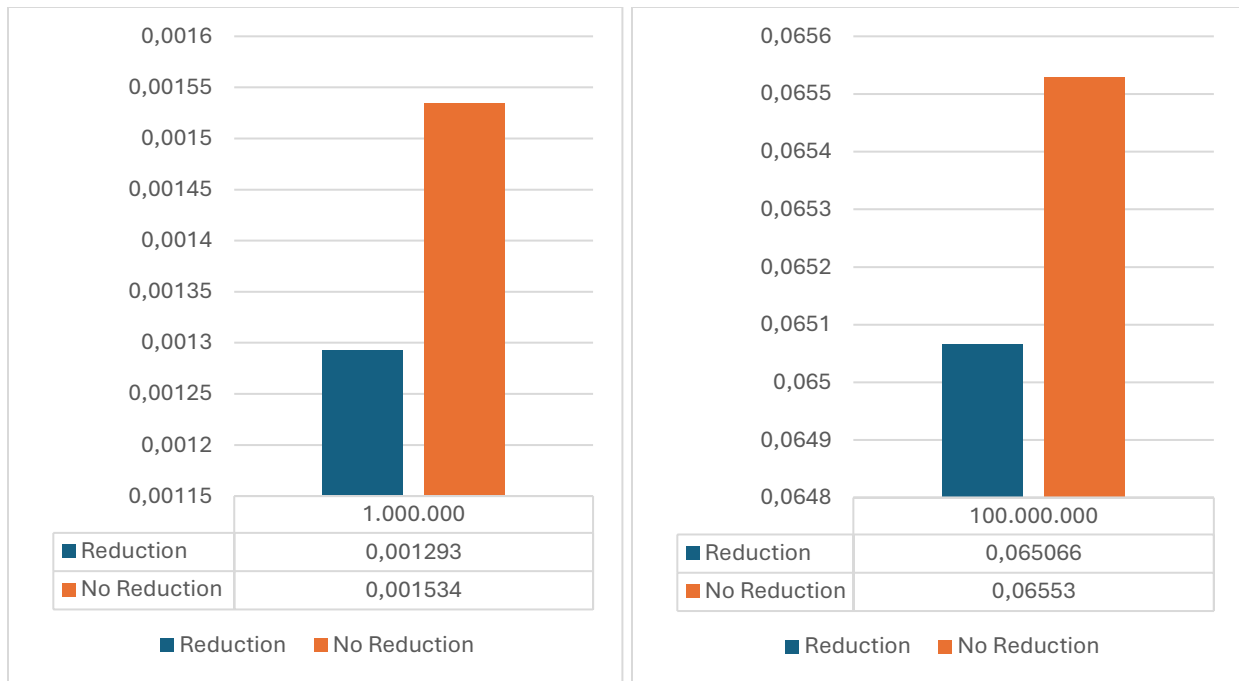
Επίδραση του Μεγέθους Προβλήματος και Thread Overhead

Μία από τις κρίσιμότερες παραμέτρους στον παράλληλο προγραμματισμό είναι ο όγκος εργασίας που ανατίθεται σε κάθε νήμα. Για να μελετηθεί η συμπεριφορά του OpenMP σε διαφορετικά μεγέθη προβλήματος, διατηρήθηκε σταθερός ο αριθμός των νημάτων στα 8 και μεταβλήθηκε ο αριθμός των υποδιαιρέσεων N . Το πείραμα διεξήχθη και για τις δύο μεθόδους συγχρονισμού.

Πίνακας Χρόνοι Εκτέλεσης 8 Threads μεταβάλλοντας το N

Μέγεθος N	Reduction (sec)	No Reduction (sec)
1.000	0,000146	0,000165
10.000	0,000066	0,000130
100.000	0,000176	0,000248
1.000.000	0,001293	0,001534
100.000.000	0,065066	0,065530





Σχολιασμός Αποτελεσμάτων:

Τα αποτελέσματα επιβεβαιώνουν τη θεωρία του διαχειριστικού κόστους. Παρατηρείται στην υλοποίηση με Reduction, η εκτέλεση για $N=10.000$ ήταν *ταχύτερη* από την εκτέλεση για το μικρότερο $N=1.000$. Επιπλέον ο καθαρά σειριακός κώδικας για $N=1.000$ ολοκληρώθηκε σε μόλις 0,000004s. Αυτό συμβαίνει διότι, σε πολύ μικρά N , ο χρόνος που καταναλώνει το λειτουργικό σύστημα για να δημιουργήσει, να κατανείμει τη μνήμη, να συντονίσει και να τερματίσει 8 νήματα υπερκαλύπτει κατά πολύ τον χρόνο του ίδιου του μαθηματικού υπολογισμού. Επίσης στα μικρά μεγέθη, η μέθοδος Reduction είναι σταθερά ταχύτερη από τη μέθοδο της χειροκίνητης τοπικής συσσώρευσης. Η εσωτερική διαχείριση της reduction από τον compiler του OpenMP εμφανίζει μικρότερο overhead κατά τη δημιουργία των τοπικών αντιγράφων. Καθώς το μέγεθος του προβλήματος αυξάνεται στο μέγιστο, ο υπολογιστικός φόρτος γίνεται τεράστιος. Σε αυτό το σημείο, η διαφορά μεταξύ των δύο μεθόδων γίνεται πρακτικά αμελητέα, αποδεικνύοντας ότι σε μεγάλα grain sizes ο κυρίως περιοριστικός παράγοντας είναι η ταχύτητα των ίδιων των πυρήνων και όχι ο τρόπος συγχρονισμού στο τέλος του βρόχου.

Μέρος Β

Στο δεύτερο μέρος της εργασίας, ο στόχος μετατοπίζεται στη μελέτη και την αντιμετώπιση του φαινομένου της ανισοκατανομής φόρτου. Για να προσομοιώσουμε αυτό το πρόβλημα, αντικαθιστούμε την απλή συνάρτηση $f(x)=x^2$ με μια συνθετική συνάρτηση υψηλού υπολογιστικού κόστους, η οποία παρουσιάζει έντονες διακυμάνσεις στο βάρος των υπολογισμών ανάλογα με την τιμή του x . Μέσω αυτής της προσέγγισης, επιδιώκουμε να αναδείξουμε τα πλεονεκτήματα του δυναμικού διαμοιρασμού έναντι του στατικού.

Προσομοίωση Ακανόνιστου Φόρτου Εργασίας

Η συνάρτηση $f(x)$ τροποποιήθηκε εισάγοντας τριγωνομετρικούς υπολογισμούς και εσωτερικούς βρόχους με μεταβλητό πλήθος επαναλήψεων. Συγκεκριμένα, χρησιμοποιήθηκε η συνάρτηση ημιτόνου $\sin(x * 123.45)$ για τη δημιουργία απότομων εξάρσεων υπολογιστικού βάρους στο πεδίο ορισμού.

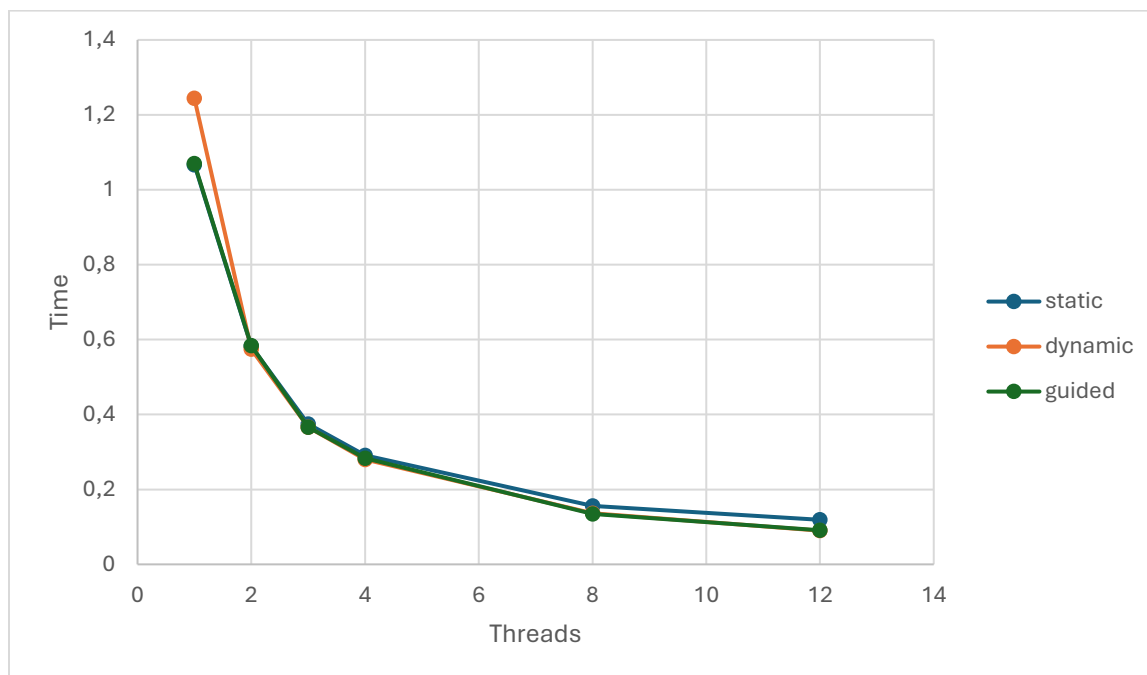
```
double f(double x) {  
    double temp = x;  
    int weight = (int)(fabs(sin(x * 123.45)) * 600);  
  
    for (int j = 0; j < weight; j++) {  
        temp = sqrt(temp * temp + 0.00000001);  
    }  
    return temp * temp;  
}
```

Πειραματικά Αποτελέσματα

Αρχικά, εξετάζεται η συμπεριφορά των τριών πολιτικών δρομολόγησης αυξάνοντας τον αριθμό των νημάτων 1, 2, 3, 4, 8, 12 διατηρώντας το μέγεθος του πακέτου σταθερό στη βέλτιστη τιμή $\text{Chunk} = 100$.

Πίνακας χρόνοι εκτέλεσης με σταθερό μέγεθος $\text{Chunk} = 100$

Threads	Static	Dynamic	Guided
1	1,067	1,244	1,070
2	0,582	0,575	0,584
3	0,375	0,366	0,366
4	0,291	0,280	0,283
8	0,156	0,136	0,135
12	0,119	0,090	0,091



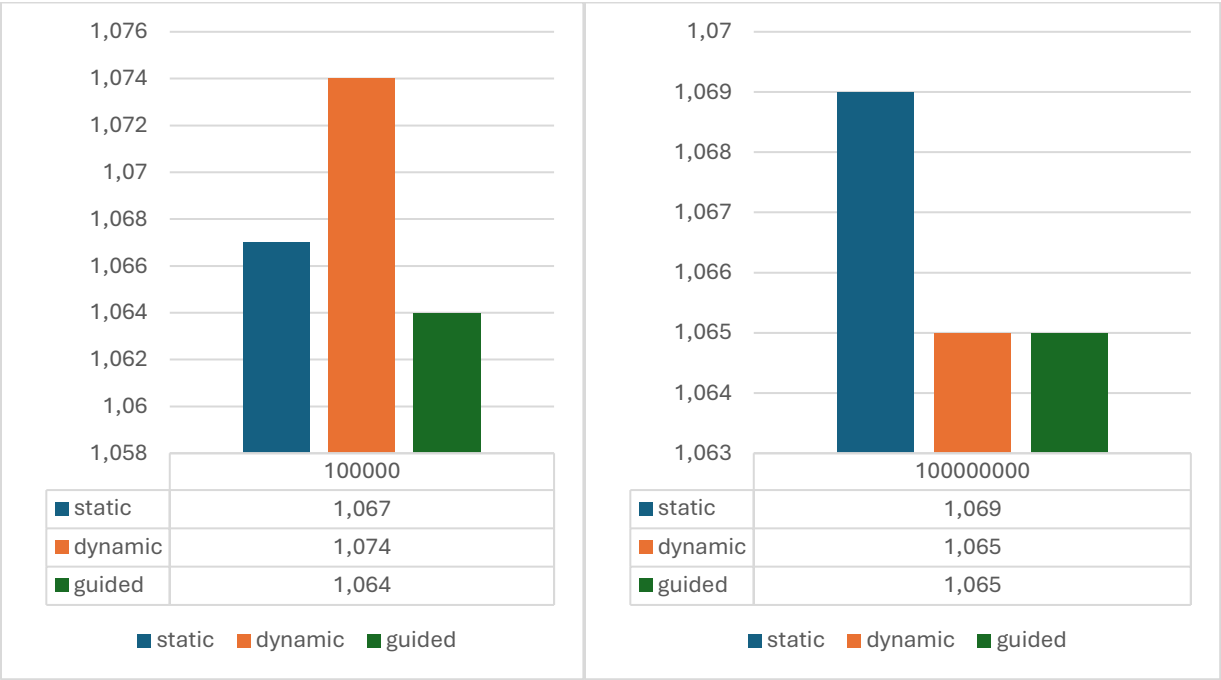
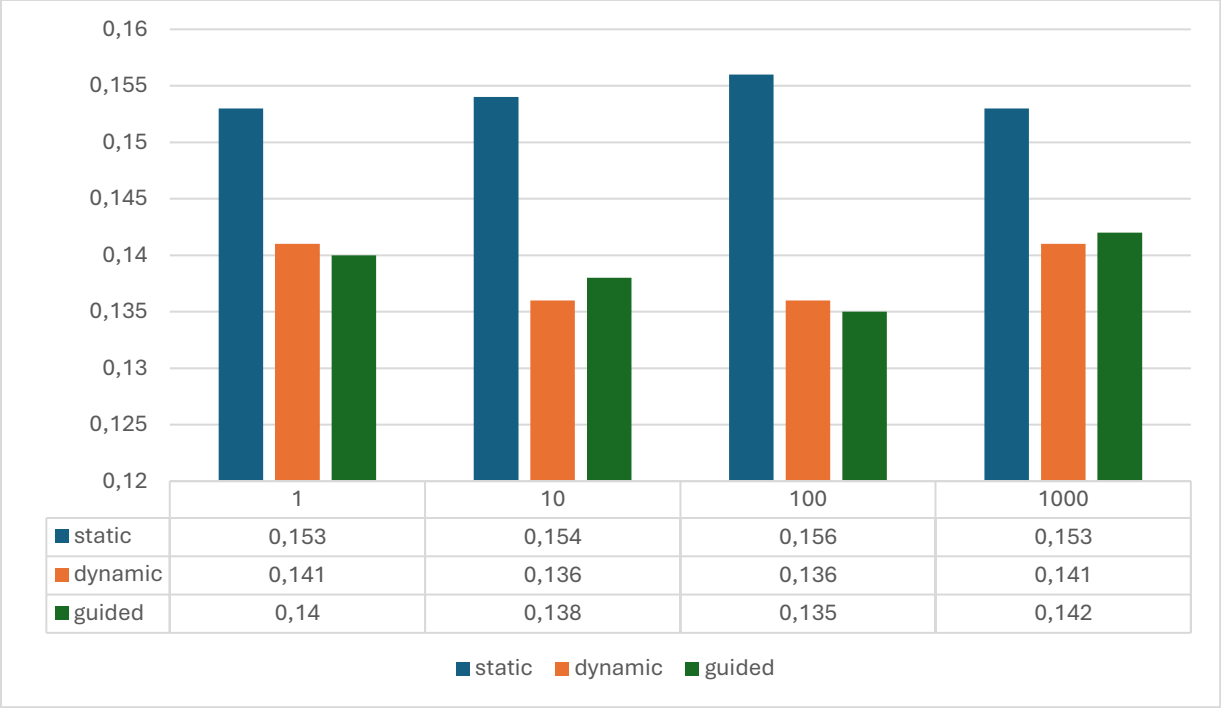
Παρατηρήσεις:

Με ένα μόνο νήμα, ο στατικός διαμοιρασμός υπερτερεί καθώς το μοναδικό νήμα επωμίζεται τόσο τον υπολογισμό όσο και το διαχειριστικό κόστος της «ουράς» στο dynamic, χωρίς να έχει άλλα νήματα για να βοηθήσουν. Στα 8 νήματα, η διαφορά είναι πλέον αισθητή, με το Guided και το Dynamic να ξεπερνούν σταθερά το Static. Η μετάβαση από τους 8 φυσικούς πυρήνες στα 12 λογικά νήματα προσφέρει μια βελτίωση στο Dynamic/Guided. Αντίθετα, το Static δυσκολεύεται. Αυτό αποδεικνύει ότι στο όταν οι λογικοί επεξεργαστές τροφοδοτούνται διαρκώς με δουλειά μέσω της δυναμικής ουράς.

Μελετήθηκαν επίσης και οι τρεις βασικές πολιτικές του OpenMP, για τον διαμοιρασμό των $N = 100.000$ επαναλήψεων αξιοποιώντας τους 8 φυσικούς πυρήνες του συστήματος, σε διάφορα μεγέθη πακέτου.

Πίνακας χρόνοι εκτέλεσης 8 Threads ανά Πολιτική και Chunk Size

Schedule	1	10	100	1000	100.000	100.000.000
Static	0,153	0,154	0,156	0,153	1,067	1,069
Dynamic	0,141	0,136	0,136	0,141	1,074	1.065
Guided	0,140	0,138	0,135	0,142	1,064	1.065



Παρατηρήσεις:

Ο στατικός διαμοιρασμός αποδείχθηκε η πιο αργή στρατηγική. Επειδή η κατανομή γίνεται τυφλά πριν την έναρξη της εκτέλεσης, τα νήματα δεν μπορούν να προσαρμοστούν στις απότομες μεταβολές του βάρους της συνάρτησης, με αποτέλεσμα τον εγκλωβισμό κάποιων νημάτων στα βαριά τμήματα και την αδράνεια των υπολοίπων. Οι πολιτικές δυναμικής ανάθεσης, όπου τα νήματα αντλούν δουλειά από μια κοινή ουρά εν ώρα εκτέλεσης, βελτίωσαν εντυπωσιακά την απόδοση. Ο καλύτερος συνολικός χρόνος σημειώθηκε από την πολιτική Guided με Chunk 100, πετυχαίνοντας μείωση του χρόνου εκτέλεσης κατά περίπου 13% σε σχέση με την καλύτερη στατική προσέγγιση.

Τα πειράματα ανέδειξαν τον κρίσιμο ρόλο του μεγέθους πακέτου, διαγράφοντας μια ξεκάθαρη καμπύλη επιδόσεων. Τα πολύ μικρά πακέτα προκαλούν υψηλό διαχειριστικό κόστος, καθώς τα νήματα σταματούν συνεχώς για να ζητήσουν από 1 επαναληψη, αυξάνοντας τον χρόνο. Το ιδανικό μέγεθος chunk 100 προσφέρει τη χρυσή τομή μεταξύ δίκαιης ισοκατανομής φόρτου και χαμηλού overhead. Ενώ τα πολύ μεγάλα πακέτα chunk 100.000 ισούνται με το σύνολο του προβλήματος N , το πρώτο νήμα αναλαμβάνει όλη τη δουλειά σειριακά, αφήνοντας τους υπόλοιπους 7 πυρήνες εντελώς ανενεργούς.

Μέρος Γ

Στο τελευταίο μέρος της εργασίας, σε αντίθεση με τον στατικό ή δυναμικό διαμοιρασμό βρόχων, τα tasks επιτρέπουν την ασύγχρονη δημιουργία πακέτων εργασίας. Ένα νήμα αναλαμβάνει να δημιουργήσει τις εργασίες, και τα υπόλοιπα νήματα από τη δεξαμενή νημάτων τις αναλαμβάνουν μόλις είναι διαθέσιμα. Αυτή η προσέγγιση είναι ιδανική για τον ακανόνιστο και δυναμικά παραγόμενο φόρτο της τροποποιημένης μας συνάρτησης $f(x)$, καθώς τα νήματα δεν δεσμεύονται από τον δείκτη του βρόχου. Υλοποιήθηκαν δύο ξεχωριστές προσεγγίσεις παραγωγής καθηκόντων μία γραμμική και μία αναδρομική.

Γραμμική Δημιουργία Tasks

Στην πρώτη προσέγγιση, ο χώρος ολοκλήρωσης χωρίζεται σε τμήματα σταθερού μεγέθους. Ο κεντρικός βρόχος τοποθετείται μέσα σε ένα `#pragma omp single`, εξασφαλίζοντας ότι μόνο ένα νήμα διατρέχει τον βρόχο και συσκευάζει τα υποδιαστήματα σε ανεξάρτητα tasks μέσω της οδηγίας `#pragma omp task`.

```

#pragma omp parallel
{
    #pragma omp single
    {
        for (int i = 0; i < N; i += CHUNK_SIZE) {
            #pragma omp task
            {
                double local_sum = 0.0;
                int end_i = (i + CHUNK_SIZE > N) ? N : i + CHUNK_SIZE;
                for (int j = i; j < end_i; j++) {
                    double x1 = a + j*h;
                    double x2 = a + (j+1)*h;
                    local_sum += (f(x1) + f(x2)) * h / 2.0;
                }
                #pragma omp atomic
                sum_linear += local_sum;
            }
        }
    }
}

```

Μόλις ένα task δημιουργηθεί, τα ελεύθερα νήματα της παράλληλης περιοχής το αναλαμβάνουν και υπολογίζουν το τοπικό του άθροισμα, το οποίο στη συνέχεια προστίθεται με ασφάλεια στο συνολικό άθροισμα με χρήση της οδηγίας `#pragma omp atomic`.

Αναδρομική Διάσπαση

Στη δεύτερη προσέγγιση, εφαρμόστηκε αναδρομική διάσπαση μέσω της συνάρτησης `task_recursive`. Το αρχικό διάστημα διαχωρίζεται συνεχώς στη μέση, δημιουργώντας νέα tasks για το αριστερό και το δεξί μισό, αντίστοιχα.

```

double task_recursive(double start_a, double h, int steps) {
    if (steps <= THRESHOLD) {
        double local_sum = 0.0;
        for (int i = 0; i < steps; i++) {
            double x1 = start_a + i*h;
            double x2 = start_a + (i+1)*h;
            local_sum += (f(x1) + f(x2)) * h / 2.0;
        }
        return local_sum;
    }
    else {
        int mid = steps / 2;
        double sum1 = 0.0, sum2 = 0.0;

        #pragma omp task shared(sum1)
        sum1 = task_recursive(start_a, h, mid);

        #pragma omp task shared(sum2)
        sum2 = task_recursive(start_a + mid * h, h, steps - mid);

        #pragma omp taskwait
        return sum1 + sum2;
    }
}

```

Η συγχώνευση των αποτελεσμάτων διασφαλίζεται με την οδηγία `#pragma omp taskwait`, η οποία αναγκάζει την εκάστοτε γονική εργασία να περιμένει την ολοκλήρωση των παιδιών της πριν επιστρέψει το μερικό άθροισμα.

Πρόληψη Ανεξέλεγκτης Δημιουργίας Εργασιών

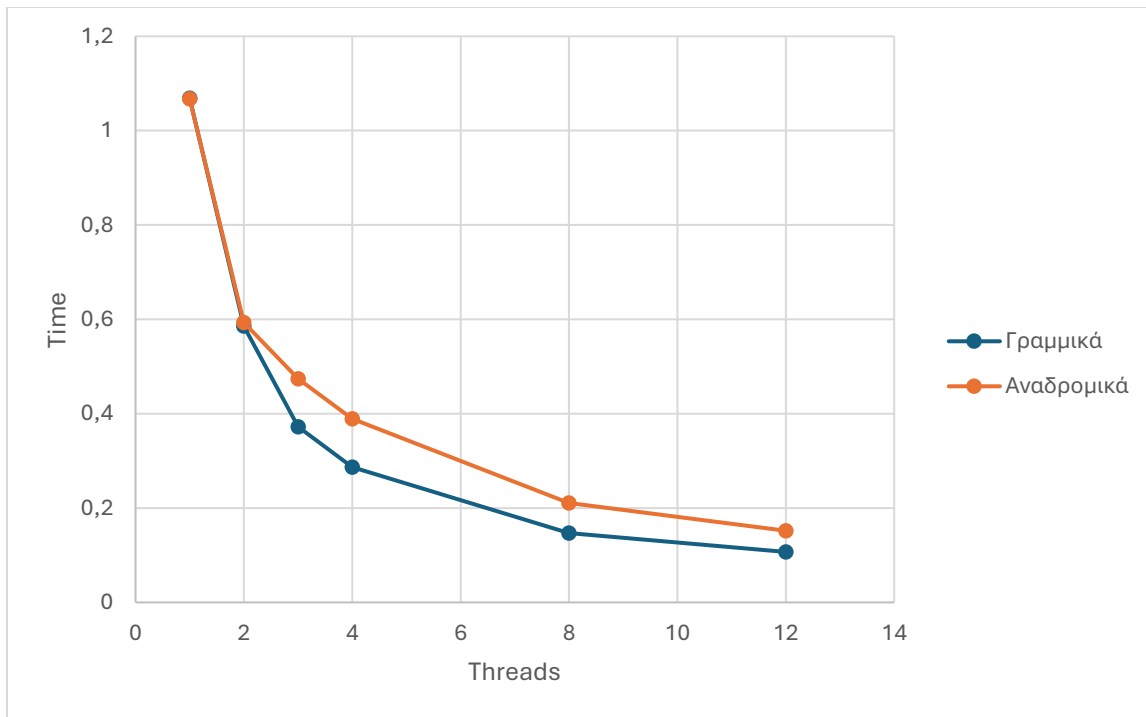
Η δημιουργία εκατοντάδων χιλιάδων tasks θα επέφερε καταστροφικό διαχειριστικό κόστος Γι' αυτό, όρισα ένα κατώφλι τερματισμού της αναδρομής. Όταν το διάστημα που ανατίθεται σε ένα task περιέχει λιγότερα από 1000 βήματα, η αναδρομή σταματά και ο υπολογισμός εκτελείται σειριακά από το νήμα που έχει αναλάβει το task. Με αυτή τη στρατηγική, διατηρείται ο αριθμός των παραγόμενων tasks σε αποδοτικά επίπεδα.

Πειραματικά Αποτελέσματα και Σύγκριση Μεθόδων

Για την αξιολόγηση των δύο προσεγγίσεων δημιουργίας καθκόντων, εκτελέστηκαν δοκιμές κλιμάκωσης 1, 2, 3, 4, 8, 12 νήματα, διατηρώντας το μέγεθος του πακέτου σταθερό.

Πίνακας χρόνοι εκτέλεσης OpenMP Tasks ανά μέθοδο δημιουργίας

Threads	Γραμμικά	Αναδρομικά
1	1,068526	1,067315
2	0,585593	0,593089
3	0,371968	0,473789
4	0,286678	0,389458
8	0,146997	0,210782
12	0,106972	0,151910



Παρατηρήσεις

Παρατηρήτε ότι σε κάθε σενάριο πολυνηματικής εκτέλεσης, η Γραμμική δημιουργία καθυκόντων υπερτερεί της Αναδρομικής. Η διαφορά στην απόδοση εξηγείται από τη φύση της αναδρομικής υλοποίησης. Στην αναδρομή, τα tasks δημιουργούν με τη σειρά τους νέα υπο-tasks. Κάθε φορά που ένα γονικό task δημιουργεί παιδιά, πρέπει υποχρεωτικά να περιμένει τον τερματισμό τους μέσω της οδηγίας `#pragma omp taskwait`. Αυτό εισάγει πολλαπλά σημεία έμμεσου συγχρονισμού, αναγκάζοντας κάποια νήματα να περιμένουν. Αντίθετα, στη γραμμική προσέγγιση, ένα μόνο νήμα γεμίζει τη δεξαμενή με ανεξάρτητα tasks, επιτρέποντας στα υπόλοιπα νήματα να τα καταναλώνουν χωρίς διακοπές και αναμονές. Αξίζει να σημειωθεί ότι ο καλύτερος χρόνος των Tasks στους 8 πυρήνες είναι ελαφρώς πιο αργός από τον καλύτερο χρόνο του δυναμικού/guided διαμοιρασμού του Μέρους Β. Αυτό δικαιολογείται, καθώς η ρητή δημιουργία και διαχείριση μεμονωμένων tasks έχει εγγενώς μεγαλύτερο διαχειριστικό κόστος από τους εσωτερικούς αλγορίθμους δρομολόγησης της `omp for schedule(guided)` του OpenMP. Παρόλα αυτά, τα tasks παραμένουν ένα ισχυρότατο εργαλείο για προβλήματα όπου η χρήση μιας απλής λούπας `for` δεν είναι καν εφικτή.

Bonus

Στο τελικό στάδιο της μελέτης, πραγματοποιείται μια συγκριτική ανάλυση της τρέχουσας υλοποίησης έναντι της προγενέστερης υλοποίησης με POSIX Threads από την 1η Προγραμματιστική Εργασία. Για να είναι η σύγκριση απολύτως δίκαιη και ουσιαστική, εξετάζεται η αντιμετώπιση της ανισοκατανομής φόρτου.

Συγκεκριμένα, συγκρίνεται η "Δυναμική Ουρά Εργασιών" με την πολιτική `schedule(dynamic)` για την ίδια ακανόνιστη συνάρτηση $f(x)$ με το ημίτονο, $N = 100.000$ και το ίδιο μέγεθος πακέτου `Chunk Size = 100`.

Πίνακας συγκριτικών χρόνων

Threads	Pthreads	OpenMP
1	1,072 s	1,244 s
2	0,611 s	0,575 s
4	0,288 s	0,280 s
8	0,139 s	0,136 s
12	0,094 s	0,090 s

Ανάλυση και Τελικό Συμπέρασμα

Από την άμεση σύγκριση των παραπάνω δεδομένων, προκύπτουν ότι οι χρόνοι εκτέλεσης μεταξύ των δύο μοντέλων είναι ταυτόσημοι σε όλα τα επίπεδα κλιμάκωσης. Αυτό αποδεικνύει ότι η ρήτρα `schedule(dynamic, 100)` του OpenMP υλοποιεί ακριβώς τον ίδιο μηχανισμό που αναπτύχθηκε χειροκίνητα στην 1η εργασία μια κεντρική ουρά πακέτων την οποία τα νήματα προσπελαίνουν με αμοιβαίο αποκλεισμό για να αντλήσουν δουλειά.

Ποιοτική Υπεροχή

Εδώ αναδεικνύεται η πραγματική αξία του OpenMP. Για να επιτευχθεί ο χρόνος στα Pthreads, απαιτήθηκε η συγγραφή δεκάδων γραμμών κώδικα δημιουργία ενός παγκόσμιου δείκτη, αρχικοποίηση και καταστροφή `pthread_mutex`, προσεκτικό κλείδωμα/ξεκλείδωμα για την άντληση του chunk, και χειροκίνητη διαχείριση της `pthread_create / pthread_join`. Αντίθετα, στο OpenMP ο ίδιος ακριβώς χρόνος επιτεύχθηκε προσθέτοντας απλώς τέσσερις λέξεις σε μια ήδη υπάρχουσα σειριακή δομή: `#pragma omp for schedule(dynamic, 100)`.

Η ποσοτική σύγκριση επιβεβαιώνει ότι το OpenMP δεν θυσιάζει σε καμία περίπτωση την απόδοση για χάρη της ευκολίας. Παρέχει επιδόσεις επιπέδου χαμηλής γλώσσας, μειώνοντας δραματικά την πολυπλοκότητα του κώδικα, τον κίνδυνο σφαλμάτων και τον χρόνο ανάπτυξης.

Τελικό Συμπέρασμα

Τα αποτελέσματα αποδεικνύουν ότι το OpenMP δεν προσφέρει μόνο μεγαλύτερη ευκολία και ταχύτητα ανάπτυξης έναντι των PThreads, αλλά καταφέρνει μέσω των εσωτερικών βελτιστοποιήσεων του compiler να παράγει ταχύτερο και πιο κλιμακώσιμο κώδικα, ειδικά σε συστήματα με μεγάλο αριθμό λογικών πυρήνων.